

Welches ML-Modell für welchen Anwendungsfall?

Kompakte Lernübersicht mit typischen Einsatzgebieten und Begründungen

Die wichtigste Frage ist nicht: „Welches Modell ist generell das beste?“, sondern: „Welche Art von Problem habe ich, welche Daten liegen vor und was ist mir wichtig – z. B. Erklärbarkeit, Geschwindigkeit oder maximale Vorhersagequalität?“

1. Zuerst die Problemart bestimmen

Problemart	Was wird gemacht?	Beispiele
Klassifikation	Eine Klasse vorhersagen	relevant / nicht relevant, Spam / kein Spam, Kündigung ja / nein
Regression	Einen Zahlenwert vorhersagen	Preis, Bearbeitungsdauer, Kosten
Clustering	Ähnliche Daten ohne vorhandene Labels gruppieren	Kundengruppen, ähnliche Dokumente, Interviewgruppen
Dimensionsreduktion	Viele Merkmale auf wenige neue Dimensionen reduzieren	Visualisierung, Daten vereinfachen, Redundanz reduzieren

2. Modelle im Überblick

Modell	Typ	Warum nutzt man es?	Typische Anwendungsfälle
Logistic Regression	Klassifikation	Sehr gute Baseline; schnell, robust und gut interpretierbar. Liefert Wahrscheinlichkeiten über <code>predict_proba()</code> .	Binäre oder Multi-Class-Klassifikation; strukturierte Daten; TF-IDF-Textdaten
Linear SVC	Klassifikation	Sehr stark bei vielen Features und sparse Daten. Besonders geeignet für TF-IDF-Textklassifikation.	Dokumentklassifikation, Spam, relevante / nicht relevante Textpassagen
Decision Tree	Klassifikation / Regression	Entscheidungen lassen sich als Regeln nachvollziehen. Sehr anschaulich, aber ein einzelner Baum kann überfitten.	Regelartige Entscheidungen, erklärbare Prototypen, kleine tabellarische Daten
Random Forest	Klassifikation / Regression	Viele Decision Trees stimmen gemeinsam ab. Meist robuster als ein einzelner Baum.	Tabellarische Daten, Churn, Risiko, Fehlerklassifikation
KNN	Klassifikation / Regression	Entscheidet anhand ähnlicher Nachbarn. Einfaches Prinzip, aber empfindlich gegenüber Skalierung und großen Datenmengen.	Ähnlichkeit kleiner Datensätze, einfache Klassifikation
Naive Bayes	Klassifikation	Sehr schnell und klassisch für Text. Funktioniert gut, wenn Wortvorkommen starke Hinweise auf Klassen geben.	Spam, einfache Dokument- und Themenklassifikation

Gradient Boosting	Klassifikation / Regression	Modelle werden nacheinander gebaut und korrigieren Fehler vorheriger Modelle. Oft sehr leistungsfähig auf tabellarischen Daten.	Risikoscores, Churn, SLA-Verletzung, strukturierte Business-Daten
Linear Regression	Regression	Einfache und gut interpretierbare Methode für numerische Zielwerte.	Dauer, Preis, Kosten, Mengen
K-Means	Clustering	Benötigt keine Zielvariable. Gruppiert Daten nach Ähnlichkeit.	Dokumentgruppen, Kundencluster, Auswahl repräsentativer Beispiele
PCA	Dimensionsreduktion	Reduziert viele korrelierte Merkmale auf wenige Hauptkomponenten.	Visualisierung, Vorverarbeitung, Reduktion vieler numerischer Features

3. Schnelle Entscheidungshilfe

Situation	Typischer Startpunkt
Ich habe Texte und feste Klassen.	TF-IDF + Logistic Regression als Baseline; danach Linear SVC vergleichen.
Ich möchte Wahrscheinlichkeiten / Thresholds verwenden.	Logistic Regression, weil predict_proba() direkt verfügbar ist.
Ich möchte ein sehr gut erklärbares Modell.	Decision Tree oder Logistic Regression.
Ich habe hauptsächlich tabellarische Daten.	Logistic Regression als Baseline; Random Forest oder Gradient Boosting zum Vergleich.
Ich möchte viele Bäume statt eines einzelnen Baums.	Random Forest.
Ich möchte sehr gute Leistung auf strukturierten Daten.	Gradient Boosting testen.
Ich möchte einen Zahlenwert vorhersagen.	Linear Regression als Baseline; danach ggf. komplexere Regressionsmodelle.
Ich habe keine Labels und möchte Gruppen finden.	K-Means.
Ich habe sehr viele numerische Features.	PCA zur Dimensionsreduktion prüfen.
Ich suche ein sehr schnelles klassisches Textmodell.	Naive Bayes oder Linear SVC.

4. Warum beginnt man oft mit einer Baseline?

In Data-Science-Projekten startet man meist mit einem einfachen Modell. Das liefert einen Vergleichswert. Erst danach lohnt sich ein komplexeres Modell.

Beispiel: TF-IDF + Logistic Regression erreicht $F1 = 0,82$. Ein deutlich komplexeres Modell erreicht $F1 = 0,83$, braucht aber zehnmal länger. Dann ist die einfache Lösung möglicherweise die bessere technische Entscheidung.

5. Typischer Modellvergleich

Bei Klassifikation sollte nicht nur Accuracy betrachtet werden. Je nach Problem sind Precision, Recall, F1, Confusion Matrix, Laufzeit und Erklärbarkeit relevant.

Kriterium	Bedeutung
Precision	Wenn das Modell eine positive Klasse meldet: Wie oft stimmt sie?
Recall	Von allen tatsächlich positiven Fällen: Wie viele wurden

	gefunden?
F1	Kompromiss aus Precision und Recall.
Confusion Matrix	Zeigt, welche Klassen konkret miteinander verwechselt werden.
Laufzeit	Wie teuer sind Training und Vorhersage?
Erklärbarkeit	Kann man nachvollziehen, warum das Modell entschieden hat?

6. Beispiele im ASM-nahen Lernkontext

Use Case	Modell	Warum?
Textpassage als relevant / nicht relevant klassifizieren	TF-IDF + Logistic Regression	Einfacher, nachvollziehbarer Startpunkt; Confidence und Thresholds möglich.
Viele Textmerkmale, Fokus auf Klassifikationsleistung	Linear SVC	Sehr passend für sparse TF-IDF-Vektoren.
Neue Interviews gruppieren, um typische Fälle und Ausreißer zu finden	Embeddings + K-Means	Keine Labels notwendig; Cluster helfen bei Sampling und Ground-Truth-Auswahl.
Tabellarische Metadaten zu Fällen analysieren	Random Forest / Gradient Boosting	Gut für nichtlineare Zusammenhänge in strukturierten Daten.
Bearbeitungsdauer schätzen	Linear Regression	Numerischer Zielwert statt Klasse.
Viele numerische Merkmale vor einer Visualisierung reduzieren	PCA	Reduziert Dimensionen und macht Muster leichter darstellbar.

7. Merksätze

- Textklassifikation: zuerst TF-IDF + Logistic Regression oder Linear SVC testen.
- Tabellarische Klassifikation: Logistic Regression als Baseline, danach Random Forest oder Gradient Boosting.
- Zahlenwert vorhersagen: Regression statt Klassifikation.
- Keine Labels vorhanden: Clustering statt supervised learning.
- Viele Features: PCA kann helfen, ist aber kein Vorhersagemodell.
- Ein komplexeres Modell ist nicht automatisch besser.
- Die Datenqualität und die Evaluation sind meist wichtiger als der reine Modellname.